## Computational geometry

*Part VII Selected Topics*

Geometric predicates, degenerate inputs, and data-structure order must agree. Orientation, intersection detection, convex hulls, and closest pairs show how local decisions produce correct global results. Prerequisites are vectors, cross products, sorting, and sweep lines.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Chapter map

The sections cover segment properties, detection of any intersecting pair, convex hulls, and closest pairs. The goals are to use cross products for turns, prove a sweep-line intersection detector, and analyze hull and closest-pair algorithms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Closed segments and orientation

A point is $p = (x, y)$. The closed segment $ab = \{a + t(b - a) : 0 \le t \le 1\}$ includes endpoints. Intersection includes endpoint contact, collinear overlap, and segments reduced to a point, as well as proper crossings.

$$\operatorname{orient}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

With the x-axis pointing right and y-axis pointing up, a positive value means a counterclockwise left turn from $a$ through $b$ to $c$. Negative means right turn, and zero means collinear. This is the signed parallelogram area. The triangle area is half its absolute value. Reversing cross-product order reverses the signs and requires reversing the turn convention.



## Exact signs and arithmetic limits

If integer coordinates have absolute value at most $B$, differences are at most $2B$, and the determinant has absolute value at most $8B^2$. The coordinate type may be too narrow for products. Widen **before** subtraction and multiplication or use arbitrary precision. A predicate on $b$-bit coordinates multiplies $O(b)$-bit integers and is not always constant machine time.

Floating-point cancellation can turn a small nonzero determinant into zero. Fixed epsilon cannot guarantee correct signs at every scale. Scaling coordinates by $\lambda$ multiplies orientation by $\lambda^2$, while the turn stays the same. Treating approximate collinearity as exact can also destroy transitivity of a sorting comparator.

Proofs here use exact real predicates. Reproducible examples use exact integers or rationals. Floating implementations need error-bounded filters followed by increased precision when the filter is inconclusive. An unproved epsilon does not replace a sign test.

## Complete segment-intersection test

Let $o_1 = \text{orient}(a,b,c), o_2 = \text{orient}(a,b,d), o_3 = \text{orient}(c,d,a), o_4 = \text{orient}(c,d,b)$. The on test requires both collinearity and closed bounds on both axes.

```
on(a,b,p):
    return orient(a,b,p)==0 and
           min(ax,bx)<=px<=max(ax,bx) and
           min(ay,by)<=py<=max(ay,by)
intersect(a,b,c,d):
    if sign(o1)*sign(o2)<0 and sign(o3)*sign(o4)<0: return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

Compare signs rather than multiplying large determinant values. Vertical segments need no slope, and on handles point segments. The bounding box alone is sufficient only after collinearity is already known.



## Why both sides must straddle

When no orientation is zero, opposite signs for $o_1, o_2$ place $c, d$ on opposite sides of the line through $ab$. Their unique line intersection lies inside $cd$. Opposite signs for $o_3, o_4$ place that same intersection inside $ab$, so the segments meet.

Conversely, an interior crossing puts each segment's endpoints on opposite sides of the other's line. Zero orientations cover an endpoint on another segment or collinearity. Two overlapping closed collinear intervals have an endpoint in the other interval, so the four on tests are complete. A point segment also reduces to testing its point on the other segment.

One straddling pair alone is insufficient. A segment may cross the extension of $ab$ beyond its endpoints.

## Crossings, contacts, overlap, and separation

Fix $a = (0, 0), b = (4, 0)$.

| c,d | o1,o2 | o3,o4 | Result |
|---|---|---|---|
| (2,-1),(2,2) | -4,8 | 6,-6 | Interior crossing (2,0) |
| (4,0),(6,2) | 0,8 | 8,0 | Shared endpoint b |
| (2,0),(6,0) | 0,0 | 0,0 | Overlap [2,4] |
| (5,0),(6,0) | 0,0 | 0,0 | Disjoint bounding boxes |

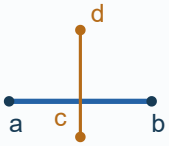
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Crossings, contacts, overlap, and separation (continued)

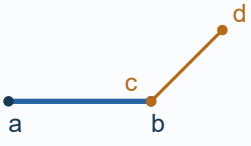
Closed-segment intersection includes endpoints and collinearity

Interior crossing




$o_1 = -4, o_2 = 8$

Shared endpoint




$o_1 = 0, o_2 = 8$

Collinear overlap




Common interval [2,4]

Collinear separation




Disjoint bounding boxes

Blue segment a=(0,0)→b=(4,0). Both axes have the same scale; y points upward.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Detect whether any pair intersects

For $n$ closed segments, return whether some pair intersects. Listing all intersections is a different task. All-pairs testing costs $\Theta(n^2)$, and an input with $\Theta(n^2)$ intersections already needs quadratic output for full enumeration.

First consider nonvertical segments. Order each segment's endpoints as left $l$, right $r$, with $l_x < r_x$. At sweep position $x$, active segments satisfy $l_x \le x \le r_x$. The status orders them by height $y_s(x)$. Events are endpoints. On insertion, test the new segment against its upper and lower neighbors. Before deletion, test the two neighbors that will become adjacent.

The key is current geometric height, not segment ID:

$$y_s(x) = \frac{l_y(r_x - l_x) + (r_y - l_y)(x - l_x)}{r_x - l_x}.$$

Denominators are positive, so compare heights by cross multiplication. With constant-cost exact predicates, each balanced-tree operation costs $O(\log n)$.



## Endpoint-only sweep detector

```
process endpoints in (x, left-before-right, y, id) order:
    if left endpoint of s:
        insert s in T at this x
        if intersect(s,below(s)) or intersect(s,above(s)): return true
    else:
        a=below(s); b=above(s)
        if both exist and intersect(a,b): return true
        delete s from T using its stored node handle
return false
```

Skip missing neighbors. Tests use entire segments, so the detector may return before the sweep reaches the intersection. It stops at the first detected pair. Replacing `return` by `print` and continuing loses correctness because status order changes after crossings.

## Earliest-intersection proof

Every true return follows an exact segment test, so there are no false positives. Suppose an intersection exists but the detector returns false. Choose an intersection $p$ of smallest x-coordinate. Before it, segments do not cross, so their height order cannot exchange and the status remains valid.

If other segments lie between two segments meeting at $p$, continuity forces those intermediate segments through $p$ too. Among this finite group, some intersecting pair is adjacent just before $p$. If contact starts at an endpoint, insertion connects the same-height group. Processing left endpoints before right endpoints retains segments ending at the same point.

The pair first becomes adjacent either when one is inserted or when the last segment between them is deleted. Both events test their entire segments and would return true, a contradiction. Collinear overlap likewise creates an intersecting adjacent pair when overlap starts at an endpoint.

No re-sort at every x is needed. Before detection, no order exchange occurs. If one is about to occur, the earlier neighbor test has already stopped the algorithm.



## Deleting a middle segment creates neighbors

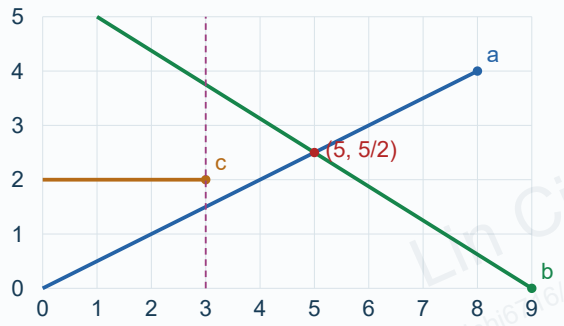
Let $a : (0, 0) \to (8, 4)$, $b : (1, 5) \to (9, 0)$, $c : (0, 2) \to (3, 2)$. Segments $a, b$ meet at $(5, 5/2)$. Segment $c$ meets neither.

| Event | Status from low to high | New test |
|---|---|---|
| x=0 insert a | a | None |
| x=0 insert c | a,c | a,c do not meet |
| x=1 insert b | a,c,b | c,b do not meet |
| x=3 before deleting c | a,c,b | a,b meet, return true |

At x=3, heights are $3/2, 2, 15/4$. Only removing $c$ makes $a, b$ adjacent. Testing insertions alone would miss this event.

## Deleting a middle segment creates neighbors (continued)

Deleting c exposes the new neighbor pair a,b




Heights at x=3:

a: 3/2

c: 2, about to be deleted

b: 15/4

Test a,b before deleting c

Accept even if the intersection lies ahead



## Vertical segments, endpoints, and cost

For general closed segments, batch equal-x events. Insert nonvertical segments starting there, process all vertical segments at that x, then delete nonvertical segments ending there. Endpoint contact remains active.

For a vertical segment with y-range $[L, H]$, find the first status segment of height at least $L$. It intersects exactly when that height is at most $H$. This is a tree `lower_bound` in $O(\log n)$, not insertion of the vertical segment as one height key. Sort same-x vertical intervals by $L$ and maintain the largest previous $H$. If the next $L$ is at most that maximum, intervals overlap. Otherwise update the maximum. Point segments belong to this closed-interval group too.

Sorting costs $O(n \log n)$. There are at most $2n$ nonvertical updates and $n$ range queries, each logarithmic. Total sorting across vertical groups is also $O(n \log n)$. The general detector uses $O(n \log n)$ time and $O(n)$ space under constant-cost exact geometric arithmetic. Cross products on $b$-bit coordinates require the corresponding additional bit cost.

## Convex hull and output convention

A convex set contains the segment between every pair of its points. For finite $Q$, $CH(Q)$ consists of convex combinations $\sum_i \lambda_i p_i$, with $\lambda_i \geq 0$ and $\sum_i \lambda_i = 1$. It is also the smallest convex set containing $Q$.

Deduplicate coordinates first. Output only extreme vertices in counterclockwise order, without repeating the first at the end. Points in the interior of a hull edge are omitted. Empty input gives no vertices. One unique point gives that point. An all-collinear set with at least two points gives just its two endpoints.

Graham scan sorts by polar angle and maintains a stack. Jarvis march finds a supporting edge at each step and depends on hull size $h$. Neither needs floating-point `atan2`. Exact orientation and squared distance comparisons suffice for ordering and collinear ties.



## Graham scan and equal-angle points

Choose $p_0$ with smallest y-coordinate, breaking ties by smallest x. Other points lie in the upper half-plane or along positive x. Compare directions $u, v$ using $\text{orient}(p_0, u, v) > 0$. Break collinear ties by squared distance from $p_0$. Keep only the farthest point on each ray. All others lie between it and $p_0$, so removing them preserves the hull.

```
S=[]
for p in [p0]+points_in_strictly_increasing_polar_angle:
    while len(S)>=2 and orient(S[-2],S[-1],p)<=0:
        pop S
    push p
return S
```

The size test avoids empty-stack access. Handle all-collinear input by returning endpoints first. For noncollinear input, the result closes to a convex polygon. Using `<=0` removes flat intermediate vertices, matching the extreme-vertex convention. Listing every boundary point needs different same-ray and final-collinear-chain handling.

## Why a popped vertex is no longer extreme

For a new point $p_i$, consider the final two stack points $r, t$. Their polar angles increase from $p_0$. If $\text{orient}(r, t, p_i) \leq 0$, then $t$ is not a new hull vertex.

If $r = p_0$, strictly increasing angles spanning less than $\pi$ give a positive turn, so no pop occurs. Otherwise translate $p_0$ to zero. Rays $r, t, p_i$ occur in order within a span below $\pi$, so $t$ is in the cone of $r, p_i$. Write $t = \alpha r + \beta p_i$, $\alpha, \beta \geq 0$. With $D = \text{orient}(0, r, p_i) > 0$,

$$\text{orient}(r, t, p_i) = (\alpha + \beta - 1)D.$$

A nonleft turn gives $\alpha + \beta \leq 1$. Thus $t = \alpha r + \beta p_i + (1 - \alpha - \beta)p_0$ is a convex combination in the triangle. Removing it preserves the new hull. Equality puts it on edge $rp_i$, still not extreme. This containment argument justifies repeated pops.



## Stack invariant and correctness

Before processing $p_i$, the stack is exactly the counterclockwise hull vertices of the processed points. One or two points establish the degenerate base case. The first noncollinear point forms a triangle.

The new point has the largest polar angle, so $p_0 \to p_i$ is a supporting edge. All earlier points lie on its right or on its line. Looking backward from the old hull's end, edges not visible from $p_i$ remain unchanged. A contiguous visible end chain is replaced by new edges. Each pop preserves the hull by the triangle proof. When $(r, t, p_i)$ becomes a left turn, $t \to p_i$ is the other tangent. The remaining convex edges and $p_i \to p_0$ bound a convex polygon.

That polygon contains the old hull and new point, while all its vertices are input points. It both contains and is contained in the new convex hull, proving equality. Induction completes correctness. Each point is pushed once and popped at most once, so the scan takes $O(n)$. Sorting dominates at $O(n \log n)$, with $O(n)$ space.

## Two numerical pop checks

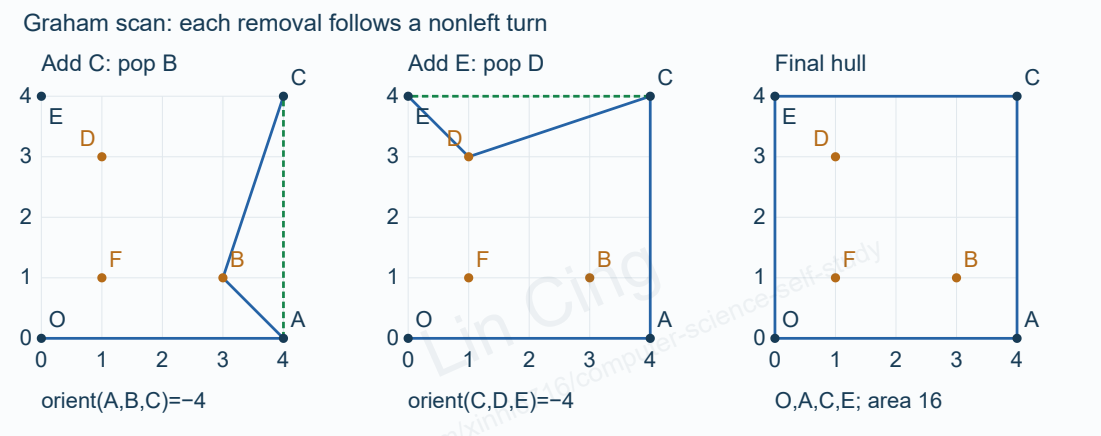
Use $O(0,0), A(4,0), B(3,1), C(4,4), D(1,3), E(0,4)$ in that angle order. Point $F(1,1)$ shares C's ray and is removed first.

| New point | Predicate | Stack after processing |
|---|---|---|
| O,A,B | orient(O,A,B)=4 | O,A,B |
| C | orient(A,B,C)=-4, pop B; orient(O,A,C)=16 | O,A,C |
| D | orient(A,C,D)=12 | O,A,C,D |
| E | orient(C,D,E)=-4, pop D; orient(A,C,E)=16 | O,A,C,E |

Indeed, $B = O/4 + A/2 + C/4$ and $D = O/4 + C/4 + E/2$. The popped points are convex combinations of other points. The hull is the square O,A,C,E, with area 16.



## Two numerical pop checks (continued)



## Jarvis march finds supporting edges

Start at the lexicographically smallest point $p$. Choose any $q \neq p$. Scan all $r \neq p$. If $\text{orient}(p, q, r) < 0$, set $q = r$. For collinear points, retain the farther one. At the end, all points lie on or left of $p \to q$, so $q$ is the next counterclockwise extreme vertex. Output $p$, set $p = q$, and stop upon returning to the start.

Why does one scan suffice? At a hull vertex, all directions to other points lie in a supporting half-plane spanning at most $\pi$. Orientation linearly orders the rays there. Each update retains the clockwise-most ray seen so far, and same-ray ties retain the farthest point. The final ray is a supporting edge. Repeated steps go between extreme vertices and do not enter the interior.

Each of $h$ hull edges scans $n - 1$ points, giving $O(nh)$ time and $O(h)$ space including output. Since $h \leq n$, worst-case time is $O(n^2)$. The square example selects A,C,E,O and has $h = 4$. Jarvis can help when $h$ is small, but is not always faster than Graham.



## Closest pair and balanced groups

For $n \geq 2$ points with unique IDs, find one pair minimizing Euclidean distance. Repeated coordinates are legal and may give zero. For fewer than two points, report no pair. Compare squared distances instead of computing roots for every candidate.

Sort X by `(x,y,ID)` and Y by `(y,x,ID)`. Split X by rank, assigning its first $\lceil n/2 \rceil$ IDs to L and the rest to R. Choose dividing x-coordinate $\ell$ between their x-ranges, allowing that same x if tied. Stably partition Y by **ID membership**. Using only $x < \ell$ would fail for equal-x inputs and could destroy balance.

Recurse for $\delta_L, \delta_R$, and set $\delta = \min(\delta_L, \delta_R)$. If zero, return immediately. Any improving cross-group pair has both endpoints within $\delta$ of the dividing line and vertical separation below $\delta$. Only the width-$2\delta$ strip matters.

## Closest-pair pseudocode

```
closest(X,Y):
    if len(X)<=3: return brute-force closest pair
    split X by rank into XL,XR; choose dividing x=ell
    split Y by IDs into YL,YR, preserving y order
    best=min(closest(XL,YL), closest(XR,YR))
    delta=distance(best)
    if delta==0: return best
    Z=[p in Y with (px-ell)^2 < delta^2]
    for i=0..len(Z)-1:
        for j=i+1..min(i+7,len(Z)-1):
            if (Z[j].y-Z[i].y)^2 >= distance(best)^2: break
            if distance2(Z[i],Z[j]) < distance2(best): best=(Z[i],Z[j])
    return best
```

The strip uses the fixed $\delta$ from the recursive results. The best distance can shrink during comparisons, but at most seven successors are inspected per point. Since only one minimum pair is requested, equal-distance candidates need no update. Strict strip membership and >= termination are valid. Listing all minimum pairs needs explicit equality and output rules.



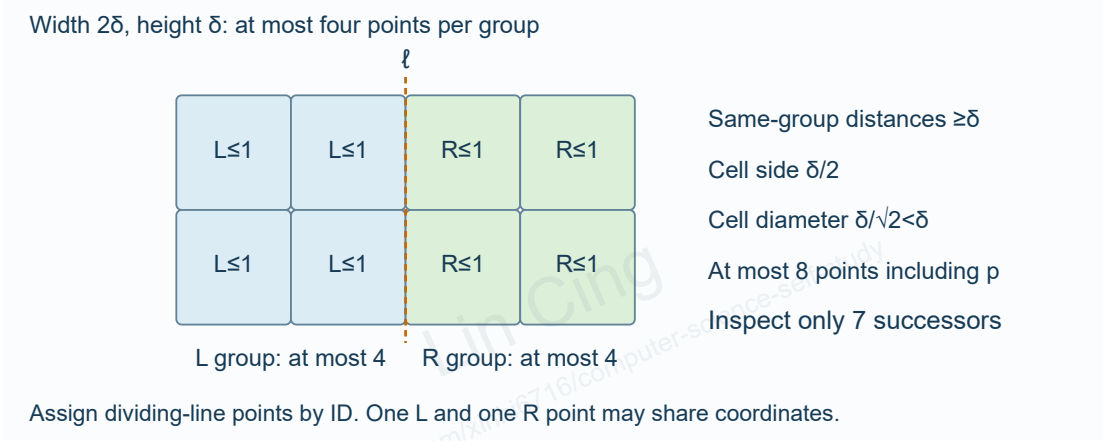
## Packing proves seven successors suffice

Fix a strip point $p$. Any later point $q$ that improves the result has $0 \le q_y - p_y < \delta$, placing it in a width-$2\delta$, height-$\delta$ rectangle. L points lie in its left $\delta$-square and R points in its right square. Within each group, pairwise distances are at least $\delta$, by recursion.

Divide each half-square into four $\delta/2$-squares. Assign boundary points using fixed half-open conventions. A cell's diameter is $\delta/\sqrt{2} < \delta$, so it contains at most one point from that group. There are at most four points per group and eight total. Excluding $p$, at most seven remain. Every possible improving $q$ is among the next seven positions in y-order.

Points on the dividing line belong to one group by ID and are counted in that group's half-square. Do not treat points from both groups on that line as if they share one within-group spacing guarantee. Cross-group duplicate coordinates are found by these comparisons. Within-group duplicates already make recursion return zero.

## Packing proves seven successors suffice (continued)






## A complete merge example

X order is $A(0,0), B(1,4), C(2,1), D(3,7), E(4,1), F(5,5), G(6,0), H(7,8)$. Split A-D from E-H and choose $\ell = 7/2$.

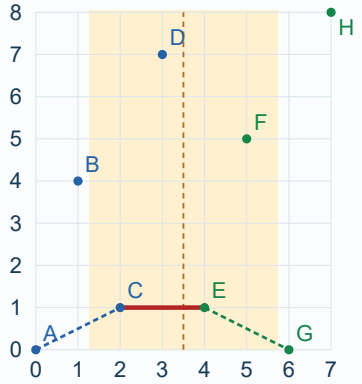
The left best pair A,C has squared distance 5. The right best pair E,G also has squared distance 5. Thus $\delta = \sqrt{5}$. The strip in y-order is C(2,1), E(4,1), F(5,5), D(3,7).

| Point | Successor checks | Result |
|---|---|---|
| C | E has squared distance 4; next F has y-gap 4 | Set best=C,E; 4≥2, stop this row |
| E | F has y-gap 4 | Cannot improve distance 2, stop |
| F | D has y-gap 2 | Equality cannot improve, stop |
| D | No successor | Done |

The cross-group pair C,E has distance 2, improving both subproblems' $\sqrt{5}$. The figure uses the fixed $\sqrt{5}$ strip even after the best distance shrinks.

## A complete merge example (continued)

Both sides have distance √5; cross-group C,E improve it to 2




ℓ=7/2, fixed δ=√5

L group A-D; R group E-H

Strip in y-order: C,E,F,D

$|C-E|^2=(4-2)^2+0^2=4$

Both sides have minimum squared distance 5

After cross-group checks: best²=4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Stable partitioning determines the bound

Rank splitting X, stable ID partitioning of Y, strip construction, and seven comparisons per point take $O(n)$ nonrecursive work. Thus

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) = O(n \log n).$$

The initial two sorts also take $O(n \log n)$. Sorting Y again in every subproblem instead gives $O(n \log n)$ work per merge level contribution and overall $O(n \log^2 n)$. Mark each ID's group at the current level. Searching the whole L array separately for every Y point would not be a linear partition.

With depth-first recursion and prompt release of temporary arrays, array lengths along one recursive chain sum to $n + n/2 + \cdots = O(n)$. Sibling arrays and return records preserve the $O(n)$ workspace bound, plus $O(\log n)$ stack. Saving every array in the entire recursion tree instead takes $O(n \log n)$ space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 33.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study